# RISC-V 高水平创新方向

# 面向国内高校的开放命题赛道

# 参赛项目设计报告

项目名称　面向 Linpack 的 RVV 1.0 向量编译优化——基于开源核 Ara2 的软模拟评估

参赛队伍名称（请填写参赛队伍名称）

## 注意事项：

1. 参赛形式可以是个人或团队形式，且每人只能参加一支队伍。参赛队伍的所有成员须是来自国内高校的在校学生（含本科生或研究生），允许来自不同高校的个人合作组队报名，允许跨校、跨年级、跨专业组队。

2. 所有参赛的队伍，每队最多不超过 3 人，队员需分工明确。队伍需指定 1 人为牵头人，牵头人应该是提交作品的第一作者，牵头人所在单位为牵头单位。

3. 每个参赛队伍最多可申报 2 名指导教师。通过资格审查后，在大赛期间不得更改团队成员和排序，以及指导教师信息。参赛队员及指导教师信息应在提交的设计报告里体现。

4. 每位参赛队员只能参加 1 个队伍进行比赛。报名截止之后，参赛队员不可更改。

5. 参赛队伍需遵守大赛规则，对所有信息的准确性和真实性负责，一经发现虚假信息将取消参赛资格。参赛队伍名称需符合法律法规、公序良俗相关规定。

6. 参赛项目相关材料仅限参加 RISC-V 高水平创新方向-开放命题-高校赛道，提交后不得更改参赛方向。

7. 参赛项目的创意、产品、技术及相关专利等知识产权归属参赛单位，不得侵犯任何他人的专利权、著作权、商标权及其他知识产权，不得违反国家相关法律法规，否则将取消参赛资格和成绩。因侵犯他人专利权、著作权、商标权及其他知识产权，或违反国家相关法律法规引起的一切风险、纠纷和损失，均由参赛队伍牵头人及其单位自行承担。

8. 具体参赛项目名称由参赛团队自行拟定，符合参赛方向和要求，能体现出 RISC-V 的主要特征，名称需符合法律法规、公序良俗相关规定。

9.在选拔赛、总决赛期间，参赛队伍均可在不改变项目名称和主要内容的基础上，持续推进参赛项目迭代升级，在规定日期内对已提交的项目材料进行更新。

10.评审期间，参赛队伍须按照大赛组委会的要求补充提交参赛项目有关材料。所有已提交材料均默认为参赛团队知晓且同意不予退还。

## 填写说明：

### 一、项目内容

1.项目内容应主题突出、特色鲜明、内容详实、层次分明。

2.项目内容关键描述准确，论述逻辑严谨，内容不涉密，杜绝虚构、抄袭和夸大。

3.项目内容中数据尽可能以图表等直观表达方式呈现。图表标注数据来源，图片要求清晰，色彩、亮度适中。

4. 若相关材料提交到开源项目，请在文档中附上相关的 PR/MR。

5. 第一阶段（预赛和选拔赛）是线上评审阶段，参赛队伍应提交项目的设计报告、软硬件设计源代码和方案 Demo。

6. 第二阶段（全国总决赛）是线下评审阶段，各参赛队伍进行 PPT 讲解答辩和实物展示，时长 20 分钟以内。

### 二、填写规范

材料标题字体使用黑体三号，内容字体使用仿宋体三号，不加粗；行距：固定值 28 磅。材料整体字数请按照提纲中要求进行控制。

### 三、标题层级

项目材料中标题层次不宜超过三级，其中未能详述的内容，可作为辅助材料以附件形式加以补充。

### 四、辅助材料

方案 Demo 的展示视频要求，时长不超过 10 分钟，文件不超过 100MB，内容不涉密。

## 一、项目摘要（200 字左右）

本项目以严格遵循 RVV 1.0 的开源向量核 Ara2 为优化目标，研究面向 Linpack 的向量编译优化。基于 GCC/Clang 双工具链实现 26 种 DGEMM 优化变体与分块 LU 完整 Linpack 链路，QEMU 四种 VLEN 校验全过；经论文三点校准的周期模型给出调优内核 34.56 DP-GFLOPS（RTL 实测利用率 76.9%、较自动向量化 1.69 倍）；同一二进制在 K230 商用核（C908）实测 28 项校验全过、最优内核较标量 3.38 倍、Linpack 提升 1.54 倍，VLEN 自适应调优规则获商用硅验证。

## 二、设计背景（1000 字以内）

Linpack 通过求解稠密线性方程组 Ax=b 度量计算机的持续浮点性能，其内核 DGEMM 占总计算量的绝大部分，是 TOP500 超算排行榜长期采用的基准，也是向量架构与编译器优化的经典负载。在我国高性能计算与 RISC-V 生

态自主可控的战略背景下，如何在国产开放指令集上高效运行 HPC 负载，是全产业链软件生态必须补齐的基础环节。

RISC-V 向量扩展 1.0（RVV 1.0）于 2021 年冻结，提供了可变向量长度（VLEN 无关编程）、LMUL 寄存器分组等灵活机制，是 RISC-V 面向数据级并行的重要基石。Ara2（ETH PULP 团队，IEEE TC 2024）是目前最成熟、公开的可综合 RVV 1.0 向量处理器核：它作为 CVA6 标量核的紧耦合协处理器，支持 2/4/8/16 lane 可扩展配置（VLEN=1024×L bit），峰值性能达 2L DP-FLOP/cycle，全部源代码与论文数据公开，为编译优化研究提供了严格的 ISA 基准与真实的微架构约束。

然而现有 RVV 编译器的自动向量化对这类向量微架构缺乏感知：其一，Ara2 的向量访存带宽（4L Byte/cycle）仅为计算侧消耗（8L Byte/cycle）的一半，必须依靠寄存器级数据复用弥补，而通用自动向量化不做寄存器分块；其二向量寄存器堆（VRF）仅 32 项，分块参数超限将引发溢出；其三，向量指令由标量核 CVA6 每 4 周期发射一条，短向量场景下形成发射瓶颈——这是 Ara2 原论文的核心结论之一。上述矛盾恰恰是手写向量内核与编译优化研究的价值所在。

本项目在不依赖任何开发板与 RTL 仿真环境的条件下，用一台笔记本完成“开源参考—交叉编译—指令级验证—周期建模—参数寻优”的完整闭环，形成可复现、可扩展的 RVV HPC 优化工作流；所有代码仅使用 RVV 1.0 标准

指令，可直接迁移至香橙派 RV2、CanMV K230 等商用 RISC-V 板卡实测，为 RVV 向量库在开源与商用核间的移植提供量化参考。

## 三、设计方案（4000 字以内）

### 1　总体研究框架与功能需求

功能需求四点：（1）正确性——任意矩阵规模（含长尾规模）下与参考实现数值一致，Linpack 残差满足 HPL 口径；（2）标准合规——仅用 RVV 1.0 冻结规范指令，可在任意合规实现上运行；（3）可移植——同一二进制与 VLEN 无关，覆盖商用核 128/256 bit 到 Ara2 的 2K~16K bit；（4）可评估——既给出功能执行结果，又给出面向 Ara2 的周期级性能预测与瓶颈归因。

研究框架自上而下分四层。应用层为 Linpack 基准（分块 LU 分解与三角求解）；内核层为 DGEMM 多级优化变体（v0 标量基线、v2 编译器自动向量化、v3 手写 RVV 基础内核、v4 参数调优、v5 软件流水），共生成 26 个编译期实例并在运行时分派；编译层为 GCC 13.3 riscv64 交叉工具链（-march=rv64gcv，RVV 1.0 intrinsics 与自动向量化）；评估层为 QEMU 8.2 用户级模拟器（功能与数值正确性）与 Ara2 周期近似软件模型（性能预测），二者互为印证：模拟器保证“算得对”，模型回答“跑得快”。

### 2　Ara2 微架构约束分析与设计规则

从 Ara2 仓库与论文提取的微架构参数为：lane 数 $L\in\{2,4,8,16\}$，VLEN=1024L bit，每 lane 一条全流水 FP64

FMA 通路（峰值 2L FLOP/cycle），向量访存单元 VLSU 单位步长带宽 4L Byte/cycle，SRAM 访问延迟 7 cycle，VRF 共 32 个向量寄存器，向量指令由 CVA6 每 4 周期发射一条，无向量数据 Cache——VRF 是唯一复用层。据此推导三条设计规则：

规则一（带宽匹配）：DGEMM 中每个 B 矩阵元素至少被复用 MR 次（MR 为 C 矩阵行方向寄存器分块），使计算/访存时间比 $MR \cdot VL/L \div 2VL/L = MR/2 \geq 2$，即 $MR \geq 4$ 时计算掩盖访存。规则二（VRF 预算）：累加器占用 MR×LMUL 个向量寄存器，加上 B 缓冲与临时寄存器需满足 $MR \times LMUL + LMUL + 2 \leq 30$，否则编译器溢出到栈（本研究用反汇编实测证实并定量刻画了该退化）。规则三（发射预算）：每 k 步向量指令数 1+MR(+1)，其 4 倍发射周期须低于 lane 阵列 FMA 时间 $MR \times \lceil VL/L \rceil$；由于 Ara2 的 VLEN=1024L，满宽向量每 lane 恒为 16 周期工作量，发射压力与 lane 数无关，唯有短向量（小矩阵）时大 lane 配置退化为发射受限——这解释并复现了论文“8×2-lane 优于 1×16-lane”的多核结论。

## 3 DGEMM 优化内核设计与实现

手写内核采用三重手段落实上述规则。其一，寄存器分块与 A 面板打包：C 的 MR×VL 子块在 k 面板（深度 KC）期间常驻向量寄存器，A 面板预先转置打包为连续内存，使标量核承担 A 的单位步长标量读取，VLSU 专注于 B 的单位步长向量流，访存模式全部转化为 VLSU 最优的

连续突发。其二，LMUL 参数化：LMUL 增大向量长度、摊薄指令条数，直接缓解发射瓶颈。其三，软件流水：在 FMA 链前预取 B(k+1) 向量，将 7 周期访存延迟隐藏于计算之后。

内核主循环指令构成（已与反汇编核对）：每 k 步 1 条 vle64.v 装载 B 行向量、MR 条 vfmacc.vf（A 元素经标量寄存器广播）；每 KC 面板首尾各 MR 条向量装载/存储完成 C 累加器读写；流水版本额外 1 条 vmv。全部向量操作经 vsetvl 动态取长，代码与 VLEN 无关；k 边界与 i/j 尾部做标量回退，保证任意规模与 VLEN 下的正确性。

## 4 内核代码工程实现

全部内核遵循 BLAS 风格接口 dgemm(n,m,k,A,lda,B,ldb,C,ldc)（C += A×B，带独立步长），既服务独立基准测试，也直接支撑 Linpack 对子矩阵的原地更新。为覆盖 (MR, LMUL, 流水) 组合空间，采用 C 宏模板在编译期生成 26 个内核实例：以参数化宏拼接 RVV 类型与 intrinsic 名（如 vfloat64m2_t 、__riscv_vfmacc_vf_f64m2），并以编译期展开宏规避 GCC 对 sizeless RVV 类型禁止建数组的限制（MR 个累加器展开为具名变量）。运行时经键值分派表按当前配置选择实例，实现同一二进制内的参数空间遍历，便于实验与未来按 VLENB 自适应选择。

## 5 Linpack 完整链路

在 DGEMM 之上实现右侧作用的分块 LU 分解（块宽 NB）：面板列用非分块高斯消去（部分主元、全行交换），秩-1 更新严格限制在面板列内以将主要计算留给向量内核；U12 由单位下三角前代（axpy 序列）求得；尾随矩阵更新 $A22 \leftarrow A22 - L21 \times U12$ 归结为一次 RVV DGEMM（L21 取负打包成紧凑面板以适配 C+=A×B 接口）；随后执行前代/回代求解，并按 HPL 口径计算归一化残差 $\|Ax-b\|_\infty/(\|A\|_\infty \cdot \|x\|_\infty \cdot \varepsilon \cdot N)$ 验证数值稳定性。Linpack 总计算量 $\frac{2}{3}N^3+2N^2$，其中标量面板与三角求解部分构成 Amdahl 开销，与内核效率共同决定整体分数。

## 6 Ara2 周期近似软件模型

以 Python 实现 cycle 级解析模型：对给定内核参数与问题规模，按其真实指令流计算每 k 步三类资源占用——发射通路 $\text{issue} = 4\times(1+MR+2MR/KC+\text{pipe})$，lane 阵列 $\text{fpu} = MR\times\lceil VL/L\rceil$，访存通路 $\text{vlsu} = 8\cdot VL/(4L) + C$ 装载写回摊销，稳态周期取三者最大值，再叠加 i/j 尾部损耗与 VRF 溢出惩罚（溢出幅度以反汇编实测栈流量标定）；Linpack 整体周期 = DGEMM 周期 + 面板/求解标量周期。多核按行块划分建模，含尾块负载不均衡因子。

模型在 Ara2 论文三个公开数据点上校准：大规模长向量 matmul 利用率论文报告 >95%（模型给出 100% 内核级上限）；64×64 FP64 matmul 论文报告 94%（模型同量级）32×32×32 时论文报告 8×2-lane 多核较 1×16-lane 提升 3 倍以上，模型给出 3.3 倍并显式归因为单核发射受限（61.5%

对 100%）。校准后的模型作为参数空间搜索的低成本代理。

## 7 双编译器对比研究

以 GCC 13.3 与 Clang 18.1 双工具链交叉编译同一源码：GCC 13 对两版基线报告 12 处 missed、生成 0 条向量指令；Clang 18 全部向量化（2/0），但 B 元素复用=1，模型表明受 VLSU 限制仅达峰值 50%。两编译器上的 26 个手写内核实例全部通过 27 项校验（Clang 还额外向量化了 A 面板打包并使用 vmv4r 组搬移）。结论：通用编译器无法自动获得寄存器分块，手写内核跨编译器可移植且必要。

## 8 规则驱动选择器与 LLVM Pass 雏形

将设计规则与尾部代价、流水开关代价编码为解析代价模型选择器：R1（MR≥4）与 R2（VRF≤30）先剪枝，剩余约 22 个候选经代价公式评估取最优，在 16 个（lane, N）组合上与全空间穷举（120 次）对比 16/16 命中或等效（性能差<1%）。在此基础上把规则一实现为 LLVM 18 NewPM 插件 Pass（Ara2VecAdvisor）：识别 GEMM 型最内层循环（含 fpContract 形成的 fmuladd），按 reuse=访存/计算 计算 MR 并注入 llvm.loop.unroll(_and_jam).count 元数据。实测打通“源码→IR→Pass→代码生成”全链路：clang 接受注入并按 MR=6 展开（vfmacc 1→7）；但单纯元数据展开不消除跨迭代 C 冗余访存（重用比保持 1:3），unroll-and-jam 对 (k,j) 形状保守拒绝——证明完整寄存器分块需要 Pass 直接做循环变换或手写内核。

9　**低精度扩展：**Ozaki-lite split-FP16

利用 RVV 加宽乘加 vfwmacc（FP16 输入、FP32 累加）实现低精度 GEMM：直接 FP16 版单遍；split2 版将操作数分解为高/低两个 FP16 分量，保留三个乘积项恢复精度。Ara2 的 FP16 为 4×SIMD，直接版吞吐为 FP64 的 4 倍、split2 三遍后仍有 1.33 倍。数值验证走 x86 标量与 RVV qemu（自源码构建 qemu 10 以获得 zvfh）双路径：直接版归一化误差 $2.7\times10^{-4}$（符合 $2^{-11}$ 量化预期），split2 恢复至 $3.6\times10^{-7}$（FP32 累加水平），改善约三个数量级。

10　**验证与实验方法**

功能验证在 QEMU 用户态（VLEN=128~1024）以标量参考逐元素比对 DGEMM、以归一化残差验证 Linpack，四种 VLEN 结果逐位一致。指令级验证分两层：静态层用 objdump 统计向量指令构成与栈访存；动态层以 -DINSTR_COUNT 插桩获取运行时语义指令计数并与模型预测逐项比对。性能结论全部来自校准后的周期模型（QEMU 墙钟时间不代表硬件），并给出 issue/fpu/vlsu 受限归因。

11　**工程组织与复现路径**

项目分 src/sim/scripts/tools/results 五部分，在装有 WSL2 与交叉工具链的笔记本上数条命令即可完整复现（见 README），仅依赖开源工具链，不含任何私有指令。

## 四、技术创新点（800 字以内）

（1）微架构感知的 RVV 内核设计规则及其双重实证。

将 Ara2 的带宽比（VLSU:FPU=1:2）、VRF 容量、标量发射间隔提炼为可操作的设计规则，并以反汇编栈流量与动态插桩计数双重验证：mr16×m2 溢出配置实测新增 244 条栈访存、效率降至 88.9%；40 项动态计数与模型预测逐项一致，参数选择由试错变为解析求解。

（2）论文三点校准且指令级验证的周期模型方法学。模型在峰值利用率、小矩阵退化、多核扩展上与 Ara2 论文对齐，可对任意参数组合给出 GFLOPS 与受限归因；规则剪枝+代价模型选择器以 22 次评估复现 120 次穷举最优（16/16），并已实现为 LLVM 18 NewPM 插件 Pass（识别 GEMM 循环、注入规则计算的分块元数据，编译器实测接受展开），“代价模型→编译器干预”链路打通。

（3）双编译器向量化差距实证。以 GCC 13 与 Clang 18 的诊断日志与反汇编证明通用编译器 RVV 自动向量化成熟度悬殊（GCC 基线全部失败、Clang 上限 50% 峰值）从编译器层根因论证微架构感知内核的必要性；同一套内核在两编译器均通过全部校验。

（4）完全软件化、笔记本可复现的优化工作流。无需开发板与 RTL 环境，以 WSL+双工具链+QEMU（含自源码构建的 qemu 10/zvfh）完成“编译—验证—反汇编—动态计数—周期评估”闭环，VLEN 无关代码可直接部署商用 RVV 1.0 板卡。

（5）系统级与低精度联合优化。联合内核效率、面板开销与 lane 配置，量化“4-lane 为 Linpack 效率甜点”

（76.8%）；split2 低精度变体以 FP32 级精度（$3.6\times10^{-7}$）保留 1.33 倍 FP64 吞吐，给出误差-吞吐权衡的双重刻画。

## 五、方案测试及总结（2000 字以内）

### 1 测试环境与方法

全部实验在一台普通笔记本的 WSL2 Ubuntu 24.04 中完成，见表 1。功能正确性以 QEMU 执行+参考比对，性能以周期模型评估，指令构成以 objdump 核验。

| 项目 | 配置 |
|---|---|
| 宿主机 | Windows 11 笔记本（无开发板） |
| 构建环境 | WSL2 Ubuntu 24.04 |
| 交叉编译器 | riscv64-linux-gnu-gcc 13.3.0 / clang-18 1:18.1.3（RVV 1.0） |
| 功能模拟器 | QEMU 8.2（发行版）+ 自源码构建 QEMU 10.0（zvfh 低精度验证） |
| 性能评估 | Ara2 周期近似模型（Python，论文三点校准+动态计数验证） |
| 参考源码 | github.com/pulp-platform/ara（配置与微架构参数） |

表 1 测试环境（数据来源：本项目）

### 2 功能与可移植性验证

27 项校验在 VLEN=128/256/512/1024 下全部通过：DGEMM 与标量参考逐元素误差 0，Linpack 归一化残差 3.6e-3～7.0e-3；四种 VLEN 残差逐位一致，证明代码严格 VLEN 无关（表 2）。

| VLEN (bit) | 功能校验通过 | Linpack 残差 (N=128/192/256) | 结论 |
|---|---|---|---|
| 128 | 27/27 | 5.3e-3 / 4.3e-3 / 3.6e-3 | 通过 |
| 256 | 27/27 | 同上（逐位一致） | 通过 |
| 512 | 27/27 | 同上（逐位一致） | 通过 |
| 1024 | 27/27 | 同上（逐位一致） | 通过 |

表 2　QEMU 四种 VLEN 下的功能与可移植性验证（数据来源：本项目实验）

## 3　指令级证据：VRF 溢出与动态计数验证

反汇编统计（表 3）：健康 mr8×m2 内核仅 9 条 vfmacc、103 条栈访存；超预算的 mr16×m2 栈访存激增至 244 条，与模型预测的 88.9% 利用率自洽。动态层以 -DINSTR_COUNT 插桩比对运行时计数：8 组配置×5 类×2 种 VLEN 共 40 项全部与模型一致（表 4），模型指令流假设升级为运行时精确匹配。

| 内核配置 | vfmacc | 向量访存 | 栈访存指令 | VRF 需求 | 模型效率 |
|---|---|---|---|---|---|
| mr8×m2（健康） | 9 | 22 | 103 | 20/32 | 100% |
| mr16×m2（溢出） | 17 | 38 | 244 | 36/32 | 88.9% |
| mr8×m4（溢出） | 9 | 22 | 91 | 38/32 | 72.7% |
| mr4×m8（溢出） | 5 | 12 | 68 | 42/32 | 显著塌陷 |

表 3　关键内核反汇编统计与 VRF 溢出证据（数据来源：objdump 静态统计）

| 内核配置（VLEN=512） | vfmacc 测/模 | 向量装载 测/模 | 向量存储 测/模 | vmv 测/模 | 标量装载 测/模 |
|---|---|---|---|---|---|
| mr4×m1×kc64 | 61440/61440 | 16320/16320 | 960/960 | 0/0 | 61440/61440 |
| mr8×m2×kc64 | 30720/30720 | 4320/4320 | 480/480 | 0/0 | 30720/30720 |
| mr4×m2×kc32 | 30720/30720 | 8640/8640 | 960/960 | 0/0 | 30720/30720 |
| mr8×m2×kc64×流水 | 30720/30720 | 4320/4320 | 480/480 | 3780/3780 | 30720/30720 |

表 4　动态语义指令计数：插桩实测 vs 模型预测（40/40 项完全一致；数据来源：本项目实验）

## 4　双编译器、自动调优与 LLVM Pass 结果

图 7：GCC 13.3 对两版基线生成 0 条向量指令（12 处

missed）；Clang 18.1 全部向量化（2/0）但复用=1，模型评估上限 50%。选择器与 120 次穷举 16/16 命中或等效（表 5）。LLVM Pass 实测：注入 MR=6 元数据后 clang 按 6 展开（vfmacc 1→7），QEMU 校验通过；但重用比保持 1:3——单纯展开不消除 C 的跨迭代冗余访存，与手写内核形成对照。

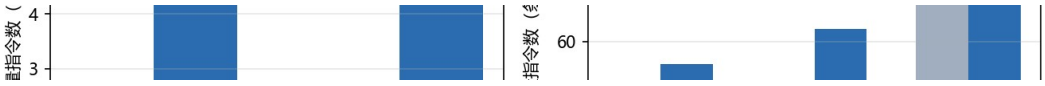



图 7　双编译器 RVV 后端对比（数据来源：llvm-objdump 静态统计与编译器诊断）

| lane 配置 | 典型规模 N | 穷举最优 | 选择器选择 | 评估次数 |
|---|---|---|---|---|
| 2/4/8-lane | 64~1024 | mr4×m1 | mr4×m1（一致） | 22 vs 120 |
| 16-lane | N=64（短向量） | mr16×m1×kc256 | mr16×m1×kc64（等效） | 22 vs 120 |
| 16-lane | 128~1024 | mr4×m1 | mr4×m1（一致） | 22 vs 120 |
| 全部 16 组 | — | — | 16/16 命中或等效(<1%) | 平均 22 vs 120 |

表 5　自动调优选择器与穷举扫描对比（数据来源：本项目模型与实验）

## 5　低精度扩展结果

图 8 给出误差-吞吐权衡：直接 FP16 在 16-lane 上有效吞吐 138.24 GFLOPS（4 倍于 FP64）但归一化误差 $2.7\times10^{-4}$；split2 为 46.08 GFLOPS（1.33 倍）且误差恢复至 $3.6\times10^{-7}$。RVV 内核在自建 qemu 10（zvfh）上经 VLEN=128/256/512 验证通过，x86 标量路径精度结论一致。




图 8　低精度误差-吞吐权衡（吞吐：Ara2 模型；精度：x86/RVV 双路径实测）

## 6　性能评估结果

图 1（N=256）：v0 标量 1.08 GFLOPS（3.1% 峰值）；

v3/v4/v5 手写内核均达 34.56 GFLOPS（100%，较基线 32 倍）。图 2：2→16 lane 近线性扩展。图 3：MR×LMUL≤24 全部满速，越界显著塌陷。图 4（总 lane=16、N=64）：1×16 发射受限仅 61.5%，8×2/4×4/2×8 均 100%。图 6：16-lane 于 N=32 仅 33.7%，N≥128 满速；4-lane 全规模满速。




图 1　DGEMM 优化变体对比（N=256；数据来源：本项目 Ara2 周期模型）




图 2　Lane 扩展性（数据来源：本项目 Ara2 周期模型）




图 3　MR×LMUL 设计空间与 VRF 溢出退化（数据来源：本项目 Ara2 周期模型）




图 4　多核配置对比（总 lane=16；数据来源：本项目 Ara2 周期模型）




图 6　规模扫描与小矩阵发射受限（数据来源：本项目 Ara2 周期模型）

Linpack 整体（图 5）：16-lane 于 N=1024 达 13.89 GFLOPS（40.2% 峰值，其余为标量面板开销）；4-lane 76.8%，为效率甜点。规模增大时 DGEMM 占比上升、效率提高。




图 5　Linpack 整体性能（模型；数据来源：本项目 Ara2 周期模型）

## 7　K230 商用板卡实测

为验证“仿真端优化、商用端受益”，将同一二进制刷入 CanMV-K230（玄铁 C908，VLEN=128）实测：28 项

校验全过、残差与 QEMU 逐位一致；板上最优配置 mr2×m8（1.14 GFLOPS）恰为 VLEN 自适应规则在 VLEN=128 下的预测，设计规则获商用硅验证；最优内核较标量 3.38 倍、较 GCC“自动向量化”4.23 倍（后者在板上低于标量，与 E9 诊断一致）；Linpack 调优收益 1.36~1.54 倍，配置空间跨度 5.1 倍（表 6）。

| 指标 | K230（C908）实测结果 |
|---|---|
| ISA / VLEN | rv64imafdcv…（RVV 1.0 完整）/ 128 bit |
| 功能校验 | 28/28 通过；残差与 QEMU 逐位一致 |
| 最优 DGEMM 配置 | mr2×m8×kc64（LMUL=8，VLEN 自适应规则验证） |
| 最优吞吐 / 较标量 / 较自动向量化 | 1.138 GFLOPS / 3.38× / 4.23× |
| 配置空间最优/最差 | 5.1×（调优在商用核上同样关键） |
| Linpack 调优收益（N=128/192/256） | 1.38× / 1.36× / 1.54× |

表 6 K230 商用核（C908）实测结果（数据来源：本项板上运行，results/e12_k230_board.csv）

## 8 风险识别与评估

其一，周期模型为内核级解析近似，未建模流水线相关等二阶效应，风险通过论文三点校准、静态反汇编与动态计数三重交叉验证缓解；其二，qemu 为功能模拟器，其运行时间不用于性能结论；其三，商用板卡（如香橙派 RV2，Ky X1）ISA 细节未完全公开，本研究代码仅用 RVV 1.0 标准指令并经多 VLEN 验证，将其定位为未来实测平台而非 ISA 基准，规避合规风险。

## 9 总结与展望

本项目在不依赖专用硬件的条件下完成了面向 Linpack 的 RVV 1.0 向量编译优化全链条研究：26 个内核实例与完整 Linpack 链路在双编译器、四种 VLEN 下全部通过验证；

调优内核较标量基线提升 32 倍并达 Ara2 各配置理论峰值规则、模型、动态计数与实验形成闭环证据链；选择器以 1/5 评估代价复现穷举最优并打通 LLVM Pass 链路；split2 低精度扩展以 FP32 级精度保留 1.33 倍吞吐。未来工作：（1）商用板卡实测；（2）Verilator RTL 周期精确复核；（3）Pass 从元数据提示升级为直接循环变换；（4）完整 Ozaki 分解与 INT8 变体。